

CONSTRA

An integrated desktop environment for geoscience interpretation

Vision & Initial Scope

Draft v0.1 · 2026-04-13

*Working name: **Constra** (subject to change)*

1. Executive Summary

Constra is a new cross-platform desktop application for geoscience interpretation. It starts from a simple observation: the tools geoscientists use today fall into two camps. On one side sit large, integrated commercial platforms such as **Petrel** that combine satellite imagery, 2D seismic, well data, and 3D reservoir models into a single interactive workspace in real-world coordinates. On the other side sit general-purpose plotting tools such as **Surfer** that are capable but manual, figure-oriented, and not designed to hold a whole project in a shared coordinate frame.

Constra aims at the underserved middle: the power of an integrated, coordinate-aware, multi-view workspace, delivered as an open, accessible desktop tool that a single geoscientist, a student, or a small consultancy can actually install and use.

This document captures the two source notes that motivated the project (summaries of Petrel and Surfer), distils them into a comparison, and then lays out the problem statement, target user, MVP scope, technology choice, and the initial backlog of tasks needed to stand the project up.

2. Background — Petrel

Petrel is a leading oil and gas industry interpretation platform. It integrates geophysical, geological, hydrogeological, and reservoir data into a single interactive environment where multiple data types can be combined and visualised together. Data can be displayed in both 2D and 3D, and every view stays in sync with the others.

2.1 Map View

All datasets are displayed in their true geographic location inside a shared Map View. Satellite imagery, seismic line locations, well locations, and cultural features (roads, buildings, infrastructure) can all be layered in the same window.



Figure 1 — Satellite basemap in Petrel's Map View. A satellite image of the project area is loaded as the bottom layer of the map, providing real-world geographic context for everything placed on top of it.

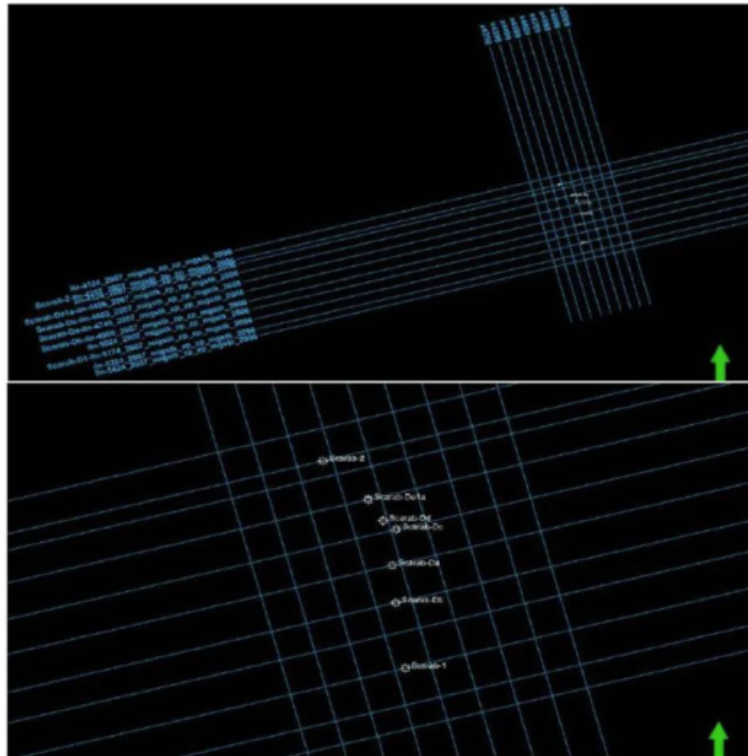


Figure 2 — Seismic lines overlaid with well locations. The same Map View now also shows the geometry of acquired 2D seismic lines together with well locations, so an interpreter can pick which line to open next based on where the wells are.

2.2 Section View

The Section View is used to display data that is naturally viewed as a vertical slice through the earth — 2D seismic lines, well logs, horizons, and faults. Selecting a line on the map opens it here as a vertical section, ready for interpretation.

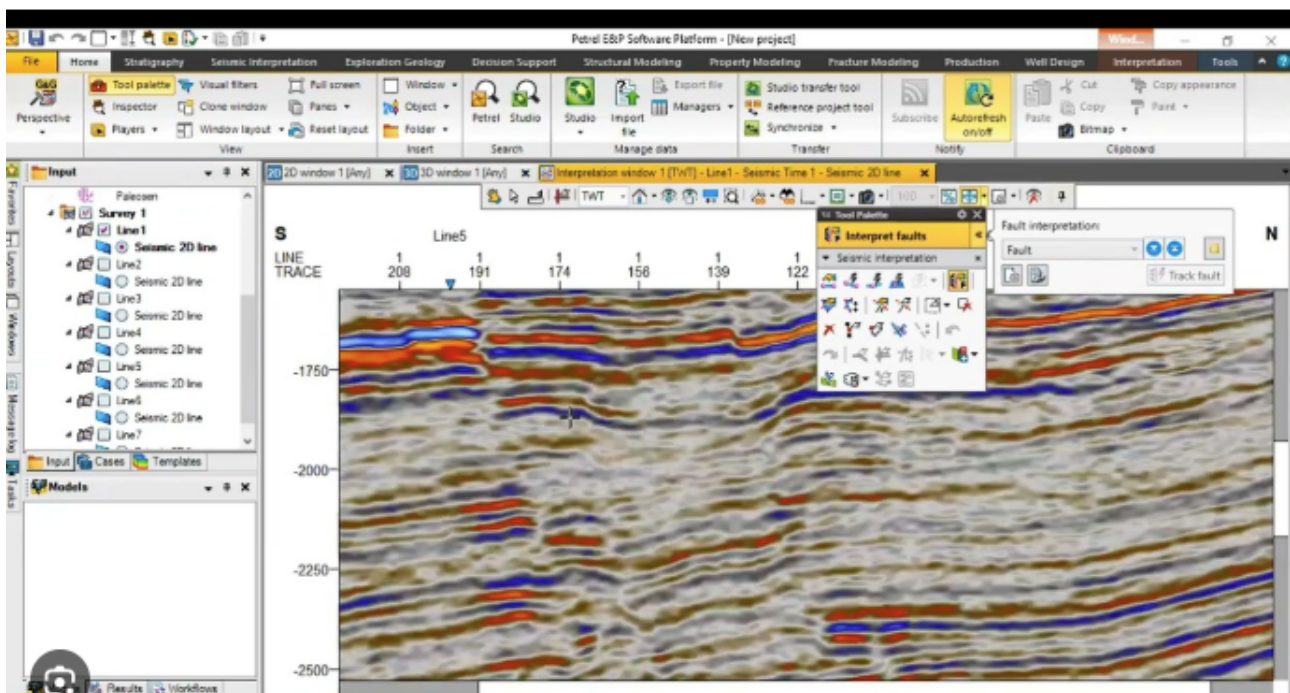


Figure 3 — 2D seismic line displayed as a vertical section in Petrel.

Key behaviour. Map View and Section View are fully interactive. An action taken in one view — selecting a line, picking a horizon, moving a crosshair — is immediately reflected in the other. The same is true for every other data type loaded into the project.

2.3 3D View

In addition to 2D, all data can be viewed, interpreted, and displayed in a 3D View, which is also synchronised with the Map View and Section View.

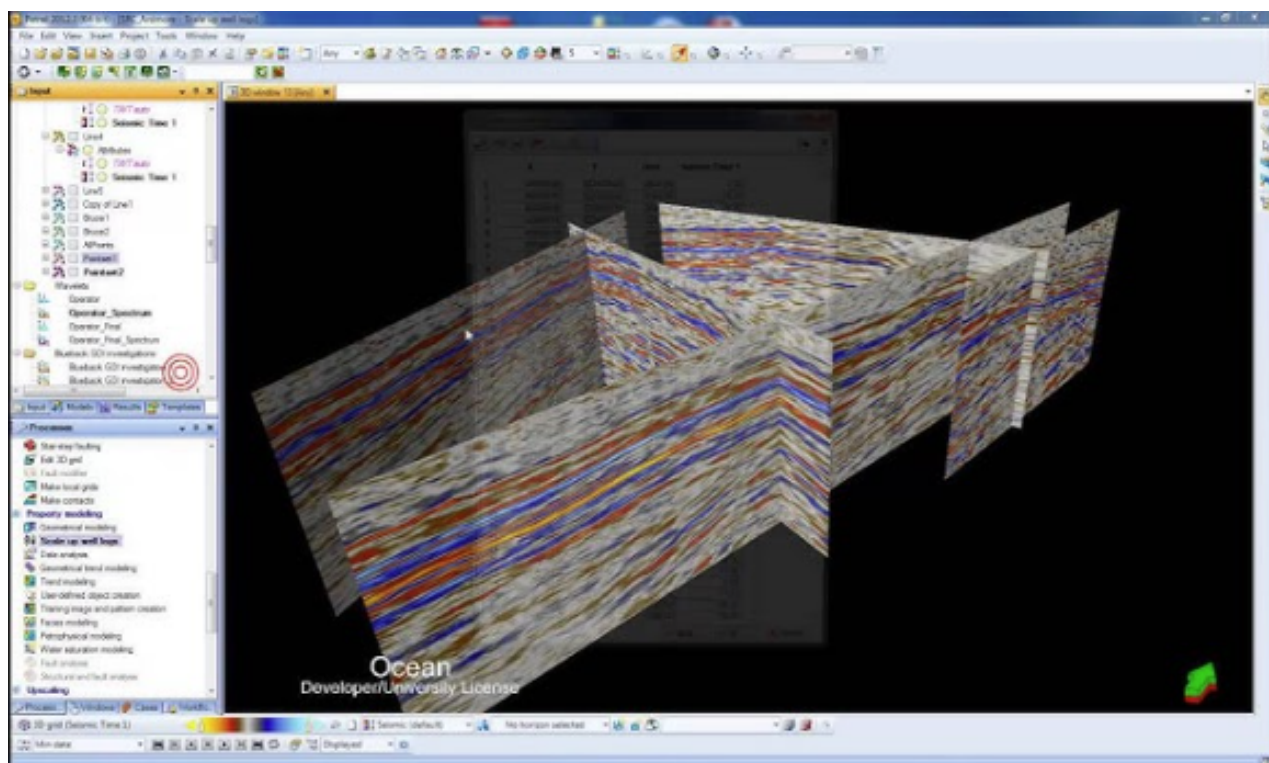


Figure 4 — 3D view of seismic lines in Petrel. Multiple 2D seismic lines are rendered together in three-dimensional space, making it easier to see how the subsurface looks along intersecting acquisition directions.

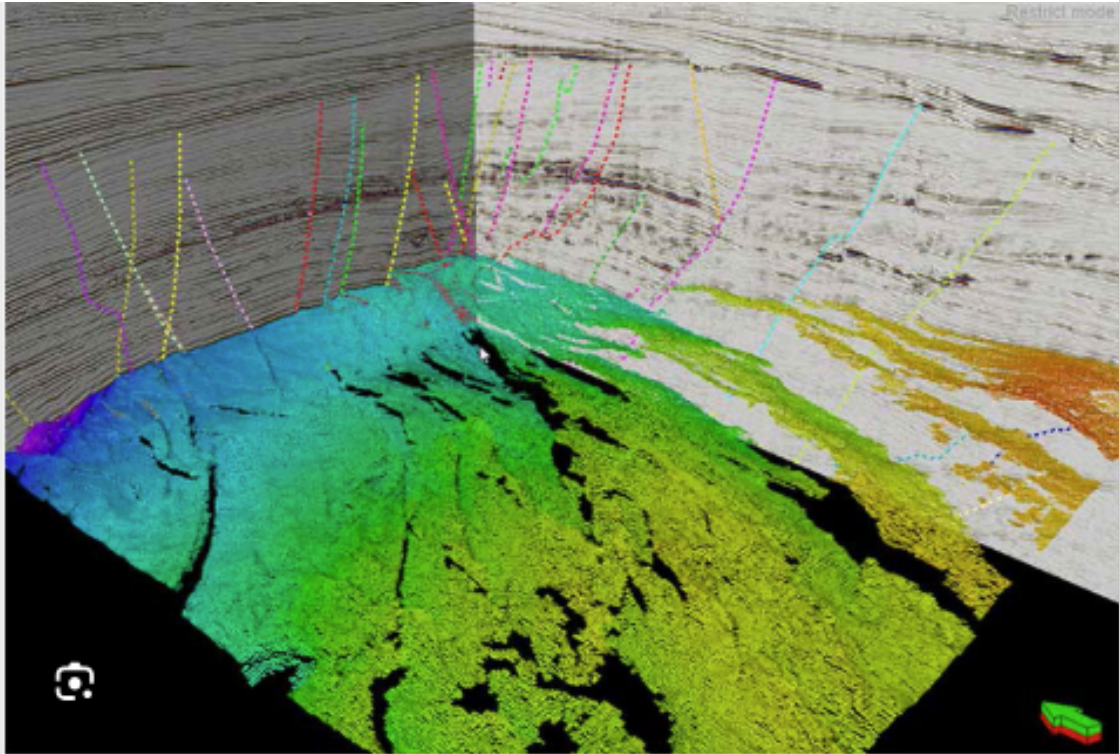


Figure 5 — 3D view of reservoir data with a well. A reservoir model is visualised in 3D alongside a well trajectory, so the relationship between the well path and the reservoir body can be assessed directly.

2.4 Coordinate System

Petrel works natively in real-world coordinates, both geographic (latitude/longitude) and projected (Easting/Northing). Every dataset is placed in its true location, and projections are handled by the software rather than by the user. This is what makes the cross-view interactivity meaningful: a well clicked in the Map View is the *same* well in the Section View and the 3D View, because the underlying coordinate system is unified.

3. Background — Surfer

Surfer (by Golden Software) is a plotting and contouring package widely used for displaying spatial data. It is capable software and well suited to producing individual map and section figures, but it was not designed as an integrated interpretation environment in the way Petrel was.

3.1 Map and Section Views

Surfer does not provide dedicated Map and Section views that are separate from each other. All output — whether conceptually a map or a vertical section — is drawn into the same generic plot window. There is no shared geographic canvas, no automatic basemap layer, and no cross-view synchronisation. Each figure is a standalone artefact constructed by hand.

3.2 3D View

Surfer offers 3D surface and wireframe plots, but these are again standalone outputs rather than a synchronised 3D workspace. A 3D plot does not share state with any 2D map or section that the user has open.

3.3 Coordinate System

Surfer does not natively place datasets into a shared real-world projected coordinate frame the way Petrel does. Data is plotted against whatever coordinate values are supplied in the input file, and the user is responsible for ensuring those values are consistent across datasets. There is no built-in projection engine that unifies every dataset into a single geographic reference.

3.4 Workflow

Surfer's workflow is predominantly manual. Every dataset, every object, every label is added to the plot by the user. This is acceptable when producing a single figure, but it scales poorly when many datasets must be kept consistent, when data is updated frequently, or when an interpreter needs to move fluidly between map, section, and 3D perspectives of the same subsurface model.

Product page: <https://www.goldensoftware.com/products/surfer>

4. Side-by-Side Comparison

The table below distils the two backgrounds into the capabilities that matter most for an integrated interpretation workflow.

Capability	Petrel	Surfer
Dedicated Map View	Yes — shared geographic canvas	No — generic plot window
Dedicated Section View	Yes — vertical slice, linked to map	No — another plot in the same window
Dedicated 3D View	Yes — synchronised 3D workspace	Standalone 3D plots only
Cross-view interactivity	Yes — actions propagate across views	No — figures are independent
Real-world coordinates (lat/long, E/N)	Native; projections handled automatically	Not native; user manages coordinate consistency
Data integration model	Unified project containing all datasets	Per-figure, assembled manually
Typical audience	Large operators, well-funded teams	Anyone producing geoscience figures
Accessibility / cost	Enterprise-priced commercial license	Commercial license, far more affordable

5. Problem Statement & Target User

5.1 The gap

A geoscientist who wants to work on subsurface data today faces a choice between two extremes. Commercial integrated platforms such as Petrel deliver an excellent multi-view, coordinate-aware workflow, but are priced for large operators and tied to heavy enterprise licensing. General plotting tools such as Surfer are affordable but produce isolated figures rather than a living, linked interpretation environment. There is no widely available, lightweight desktop tool that offers the *workflow* benefits of the integrated model (shared coordinate frame, linked views, data-type awareness) without the enterprise baggage.

5.2 Who Constrax is for

- Independent geoscientists and small consultancies who cannot justify a Petrel license but need more than a plotting tool.
- University students and researchers learning interpretation workflows on real data.
- Adjacent disciplines (shallow geothermal, mineral exploration, hydrogeology, near-surface geophysics) whose data looks like oil and gas data but whose budgets do not.
- Anyone who wants to load a GeoTIFF basemap, drop well locations on it, open a seismic section linked to those wells, and view the whole thing in 3D — without a week of manual plot construction.

5.3 What Constrax is explicitly not

- Not a full Petrel replacement. Petrel is a multi-decade, multi-hundred-person platform; Constrax will not match its feature breadth and should not try to.
- Not a reservoir simulator. Constrax is an interpretation and visualisation environment, not a numerical solver.
- Not a web application (initially). The first release is a desktop app.

6. MVP Scope

The MVP has to be small enough to ship and sharp enough to prove the core thesis: **shared coordinate frame, linked views, real geoscience data**. Everything below is in scope for v0.1. Everything else is deferred.

6.1 In scope for v0.1

- **Project container.** A Constra project is a folder that holds datasets plus a project-level coordinate reference system (CRS).
- **Map View.** 2D canvas that renders all datasets in their real-world projected coordinates, with pan, zoom, and layer visibility controls.
- **Section View.** Dedicated window for displaying one vertical section at a time, linked to the Map View.
- **3D View.** A minimal 3D scene showing the same datasets in space, linked to the Map View's selection.
- **Data formats, read-only for MVP:** GeoTIFF (raster basemap); shapefile and GeoJSON (vector features); CSV (point data such as well heads); LAS (well logs, 1D); SEG-Y (2D post-stack seismic).
- **Coordinate system support.** Per-project CRS, with on-the-fly reprojection of loaded datasets via PROJ (pyproj).
- **Cross-view selection.** Selecting a well on the map highlights it in the section view and the 3D view.
- **Windows installer.** A one-click MSI or exe for Windows 10/11.

6.2 Deferred past v0.1

- Horizon picking, fault interpretation, attribute generation.
- 3D reservoir grids and property modelling.
- Write support for any file format (v0.1 is read-only).
- Collaboration, cloud sync, multi-user projects.
- Mac and Linux installers (possible later; Windows is the priority).
- Python scripting / plugin API (likely v0.3 or later).

7. Technology Choice

7.1 Chosen stack

- **Language:** Python 3.11+
- **GUI framework:** PySide6 (Qt 6 bindings, LGPL)
- **3D and scientific visualisation:** VTK (the library behind ParaView and 3D Slicer), optionally via PyVista for ergonomics
- **Coordinate systems:** pyproj (bindings to PROJ)
- **Raster I/O:** rasterio (GeoTIFF, etc.)
- **Vector I/O:** pyogrio or Fiona for shapefile/GeoJSON; geopandas for in-memory manipulation
- **Seismic I/O:** segyio for SEG-Y
- **Well log I/O:** lasio for LAS
- **Packaging for Windows:** PyInstaller or briefcase, producing a standalone installer
- **Testing:** pytest, with pytest-qt for GUI tests
- **Linting / formatting:** ruff, black

7.2 Why this stack

This is the same toolchain that underpins ParaView and 3D Slicer, two of the most successful open-source scientific visualisation applications in existence. Every hard problem on Constr's roadmap — large raster display, 3D scene management, coordinate reprojection, seismic parsing, well log parsing — already has a mature, well-maintained Python library. Python also keeps iteration speed high, which matters enormously at v0.1 when the biggest risk is *building the wrong product*, not *building it too slowly*. A future port of performance-critical pieces to C++ or Rust remains possible; starting there would simply slow the project down before the core product ideas have been tested.

7.3 Platform strategy

v0.1 targets **Windows 10/11** only, because that is where the target users already run Petrel and Surfer. Because the chosen stack is cross-platform by construction, Linux and macOS builds are expected to come essentially for free once the Windows build is stable — but they are not a v0.1 commitment.

8. Initial Task List

The backlog below is organised into milestones. Each milestone is meant to end in a runnable application that does a little bit more than the previous one.

M0 — Project bootstrap

- Create GitHub repository (public) and add MIT license, README, .gitignore.
- Set up Python project skeleton (pyproject.toml, src/ layout, virtual environment instructions).
- Add ruff + black + pytest + pytest-qt and a minimal CI workflow (GitHub Actions) that runs them on push.
- Add a hello-world PySide6 main window that launches and closes cleanly.

M1 — Project container and CRS

- Define the on-disk Constrax project format (project folder + project.json or project.toml).
- Implement project create / open / save.
- Implement a project-level CRS setting (pick EPSG code at project creation, stored in project file).
- Wire pyproj for on-the-fly reprojection of any loaded dataset into the project CRS.

M2 — Map View

- Build the Map View widget (Qt Graphics View or a VTK 2D render window).
- Load a GeoTIFF via rasterio and display it reprojected into the project CRS.
- Load a shapefile/GeoJSON via pyogrio and overlay it.
- Load a CSV of point data (e.g. well heads) and plot each point with a label.
- Add pan / zoom / layer visibility controls and a simple layer panel.

M3 — Section View

- Build the Section View widget.
- Implement a SEG-Y reader (segio) that loads one 2D post-stack line.
- Display the seismic section with configurable colour map and gain.
- Link Section View to Map View: clicking a line on the map opens it in the section view.

M4 — 3D View

- Build the 3D View widget using VTK / PyVista.
- Render the loaded basemap, well heads, and seismic line in 3D space.
- Link 3D View selection to Map View and Section View (shared selection model).

M5 — Well logs and first installer

- Load LAS files with lasio and display logs in a small embedded log track beside the Section View.
- Package the app for Windows with PyInstaller (or briefcase).
- Produce a signed installer and publish it as a GitHub release artifact.

Shipping M5 means Constrax v0.1 is a real application: it can open a GeoTIFF basemap, plot wells on it in real-world coordinates, open a linked SEG-Y seismic line in a section view with its well logs alongside, and show the whole scene in 3D — installable from a single Windows installer. That is the smallest possible product that proves the thesis.